Handling Rollbacks with Separated Response Control Service for Microservice Architecture

Mustafa Gördesli
Department of Computer Engineering
College of Engineering and Natural Sciences
Maltepe University
Istanbul, Turkey
mgords@gmail.com
ORCID: 0000-0001-9953-419X

Ahad Nasab
The University of Chattanooga
Dept. of Engineering Management and Technology
College of Engineering and Computer Science
Chattanooga, TN, US
ahad-nasab@utc.edu
ORCID: 0000-0001-6069-0822

Asaf Varol
The University of Chattanooga
Dept. of Engineering Management and Technology
College of Engineering and Computer Science
Chattanooga, TN, US
asaf-varol@utc.edu
asafvarol@maltepe.edu.tr
ORCID: 0000-0003-1606-4079

2022 3rd International Informatics and Software Engineering Conference (IISEC) | 978-1-6654-5995-2/22/$31.00 ©2022 IEEE | DOI: 10.1109/IISEC56263.2022.9998226

***Abstract*—Working with large scale microservice based applications may be hard to maintain and control. The services in those applications must be isolated from each other and they must work in harmony. There must be an intelligent transaction management system to implement this kind of collaborative work of microservices. There are two main methodologies for transaction management, namely Two Phase Commit (2PC) and Saga pattern. There are two phases in 2PC that prepare phase and commit phase. In Saga pattern, there are two ways of implementing Saga pattern; choreography based, and orchestration based. When there is a problem in the whole transaction, controlling the rollback scenario may be hard because each microservice has its own rollout and rollback solutions in both ways of Saga pattern. If we separate the rollback system from the microservices, we might have simpler microservices and microservices that can run in parallel. So whole system works safer and faster. In this work we focus on parallel execution of microservices with separate rollback system that controls the whole application in error condition.**

Keywords—Microservices, REST, Event Driven Architecture, ACP, 2PC, Saga Pattern

I. Introduction

Microservices have become very popular for developing complex and large systems. Many companies like Amazon, Netflix use microservices to develop their services because of its benefits [1]. The applications that use microservices become fault tolerant. Also, deploying inner services becomes easier than monolithic architecture because there are separated and inner microservices that can be deployed independently [2, 3]].

Although microservice architecture brings many benefits, it has some challenges to handle. For example, maintaining data consistency is harder than monolithic architecture. Because there is only single application on monolithic architecture therefore data flow can be easily handled. On the other hand, controlling data consistency becomes harder for the whole application because there are many inner microservices [4]. Two-phase commit (2PC) and Saga pattern handle this data inconsistency problem for microservices [5]. In 2PC, the whole transaction must be done in one ACID transaction [6]. Saga pattern controls data transaction across the microservices and maintains data consistency by publishing messages between microservices that trigger next transaction [7].

If we need a cloud ready, fault tolerant system, we need to address the data consistency problem. Otherwise, the system cannot be used properly and there may be unrecoverable conditions in production systems. 2PC and Saga pattern help users handle this challenge in building services and systems.

In this work, we will explain the basics of transaction management like atomic commit protocols (ACP), ACID principles and explain 2PC and Saga patterns. We will also focus on implementing a new way of transaction controlling mechanism in Saga pattern.

II. Background

A. Two Phase Commit (2PC)

Controlling of a transaction in a system that uses many microservices is getting harder when the number of microservices increases. The ACP ensures that atomicity property of a transaction is satisfied in a system. 2PC is one of ACP and it is the base for all other ACPs [8].

ACID is a set of properties that guarantee the reliability of a transaction. ACID stands for atomicity, isolation, consistency, and durability which are defined below [9].

1. Atomicity means data consistency. When updating multiple databases, all database update transactions are either executed successfully or not.
2. Consistency refers to data integrity. The system should be in consistent state whether a transaction is successful or not.
3. Isolation means independency of transactions. All transactions continue their flow without affecting each other.
4. Durability means recoverability. When a transaction fails, all services must return to their previous states without any error.

There are prepare and commit phases in 2PC. There is a controller in 2PC that controls inner microservices. The controller of the other services sends a *prepare* request to inner microservices then waits for the acknowledgement from inner microservices. After that, the controller either sends a *commit* or *reject* (rollback) request to services [10]. “Fig. 1” represents 2PC transaction management.

978-1-6654-5995-2/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.

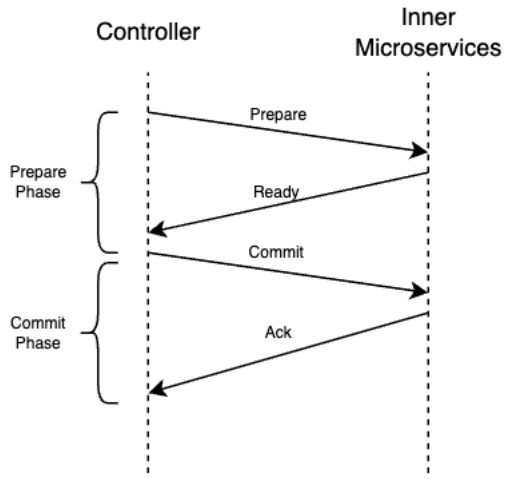


Fig. 1. 2PC Transaction Management

Controller handles the data consistency management by making commit request after all services sends ready response. It waits on prepare phase to get a response from all services. If there is at least one error on prepare command, the whole transaction fails. However, since there is no commitment on database, data consistency is still satisfied. Same as prepare phase, the controller sends commit command after all services are ready and waits for the acknowledgement. If there is an error on commit phase, there should be a rollback scenario to cover the error.

2PC tries to satisfy ACID principles, so 2PC provides a solution for data consistency. Also, there are some disadvantages of 2PC [7] [11]. The advantages and disadvantages of 2PC are listed below.

Advantages,

- Waiting for response of prepare and commit phases makes the whole system data consistent.
- Controlling of a single unit provides ease of use.

Disadvantages,

- While controller waits for response of prepare and commit phases, the system locks so there might be performance problems.
- If an error occurs on any of inner microservice on any phase, there must be a detailed rollback scenario.
- Using only one controller can become single point of failure.

B. *Saga Pattern*

Saga pattern is described as a series of operations to perform a business need and every process in a single service is processed one by one [12]. In Saga pattern, many local transactions run in many services to achieve a unit of work. Every transaction may update an independent database and publish an event that contains the response either completed successfully or not. When a transaction fails, Saga pattern executes a series of rollback steps to undo the changes.

In Saga pattern, it is guaranteed that either whole operation completes successfully or in case of failure in a single transaction, a series of rollback steps are executed. So, regardless of any errors , at the end of the process data consistency is satisfied [12].

There are generally two approaches in Saga patterns; choreography and orchestration. In choreography approach, there is no need for any external controller to handle the whole process. However, in orchestration, there must be a controller to control the transaction management [7].

a. *Choreography Approach*

In this approach there is no need for any controller to handle transaction management because every service triggers its response to the queue. Every service starts its own process when previous service triggers a successful event to the queue. For instance, if there is an error in the middle of a transaction between inner microservices, the service that has the failure, triggers an unsuccessful event to the queue, then, other services get the unsuccessful message from queue to start a rollback process and undo the changes and turn the system to its initial conditions [13].

The main advantage of using choreography approach is not having a controller. In this approach every service is acts individually and takes care of its own data consistency. On the other hand, not using a controller may be a disadvantage, because data flow may be complicated. When debugging the system, it may be hard to follow the transactions between microservices. Fig. 2 represents a successful orchestration Saga approach diagram.

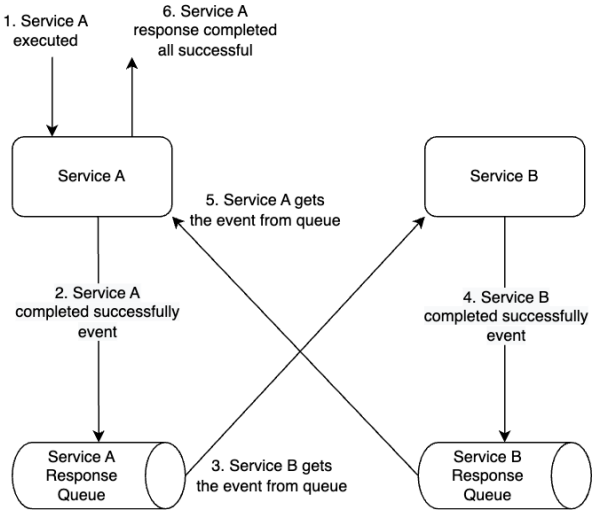


Fig. 2. Successful Choreography Saga Approach

In Fig. 2 service A gets the initial request at step 1 and processes the data and sends the successful event to its queue at step 2. At that time, service B which subscribed to service A response queue, gets the successful event, and starts its process at step 3. Service B makes the process and sends the successful event to service B response queue at step 4. service A which subscribed to service B response queue, gets the successful event at step 5. Finally, service A responds by sending the final response to the user at step 6. Only two microservices are involved in all the steps listed above. As noted by the number of steps, , when the count of microservices increases, controlling the transaction flow gets more involved.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.

In orchestration approach, because there is no controller that handles transaction management, first service may return the final response. In Fig. 2 service A gets the initial request and returns the final response. This solution may cause some undesirable conditions because service A takes the role of a contact point to the user. Instead, a response listener may be implemented to take care the contact point.

b. Orchestration Approach

In orchestration approach, a controller service is used to control and manage the transaction flow. The main difference from choreography approach is the availability of the controller service [13]. A message queue is also used in this approach. The controller service sends the transaction to the queue and related inner microservice takes the message and processes its data and sends the response to the queue. Controller service takes the response and either continues to the process by sending a message to the queue or starts the rollback process. Fig. 3 represents a successful orchestration Saga approach.

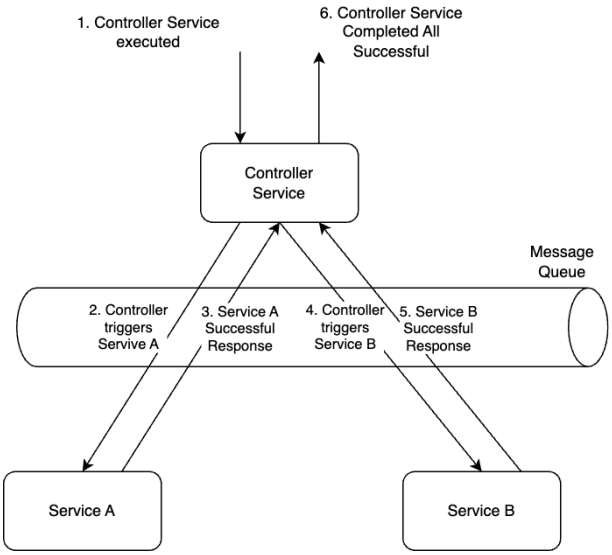


Fig. 3. Successful Orchestration Saga Approach

In Fig. 3 controller service gets the request from the user at step 1 and sends the request to service A using message queue at step 2. Then service A gets the requests, processes the request and responds back to the queue at step 3. Controller takes this response and if it is successful it triggers service B by sending the request to the queue at step 4. Service B gets the request, starts its process and then sends the response to the queue at step 5. Finally, controller sends back the final successful response to the user if all microservices have completed their processes successfully at step 6. If there is any error during this life cycle, controller starts a rollback process to satisfy data consistency.

Using one controller input point counts as an advantage, because only the user who interacts with the system knows the controller service. Also, inner microservices do not communicate with each other using message queue, instead, the controller service handles this communication. Additionally, debugging and tracing the data flow is easier in orchestration approach because developers follow only the controller service transaction flow.

III. Separated Response Control Service Design

Regardless of which pattern is used, a rollback scenario must be implemented to the system. The rollback scenario keeps the system consistent and fault tolerant. If we face an error during the communication of microservices or inside a microservice, we need to start a proper rollback scenario to turn back the system before to the state before the transactions started. So, in every step of the transaction, rollback scenarios may change since every step may be different when a transaction flows through the microservices.

In choreography approach, when a microservice throws an error, it publishes the error response to the queue, then other microservices understand that there is an error in the system and rollback scenarios may be executed. As an example, let's assume that there are 4 microservices in a system A, B, C, and D, which run respectively. When there is an error in microservice C, then A and B get the error response of C from the queue and execute the rollback scenario of their processes.

In orchestration approach, the controller service gets the error response from any of the inner microservices and then cancels the remaining transactions and starts the rollback scenario for microservices that were running previously. If the rollback is successful for all microservices, the controller service responds back to the user that there was an error and the process was rolled back successfully. Handling, controlling, and debugging the rollback system is easier in orchestration approach than in the choreography approach Because there is one point of contact which is the controller.

In this work we suggest a new approach that handles the rollback mechanism that is named *response control service*. In the Saga pattern, a microservice or a controller needs to apply the rollback, but we can add a new separated microservice that checks the error responses from the queue and applies the rollback scenario if there is any error. In Saga pattern, when an event comes from a user, all microservices run their processes respectively and wait for the request which starts the process. Even tough microservices execute asynchronously, microservices get their start trigger from the queue in an order. But when separated response control service is used, all microservices which are not dependent on each other, can start their processes at the same time. So, the system becomes faster.

Response control service subscribes to all response messages from all microservices in a system. If all microservices complete their processes successfully, they publish their successful response messages to the queue with the same unique identifier and response control service understands that a transaction completed successfully and sends the successful response to the user or the queue. When an error occurs in the transaction flow, response control service gets the unsuccessful event and executes the rollback for microservices that need to be rolled back and triggers the unsuccessful response to the user. Because all transaction flows have the same unique identifier, response control service can control and manage the responses. Also, the system with separated response control service can be implemented on either choreography or orchestration approach. The only difference is the existence of rollback service. Fig. 4 represents a system with parallel executed microservices and separated response control service for handling unsuccessful response.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.

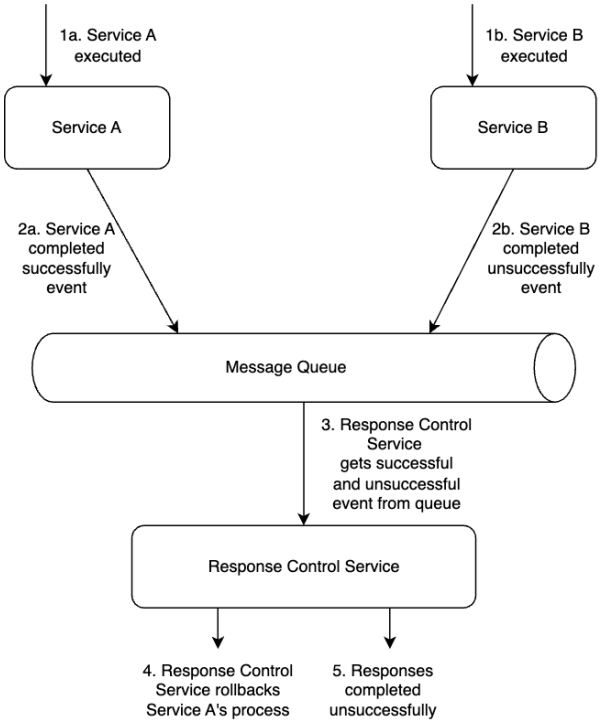


Fig. 4. Separated Response Control Service Design

In Fig. 4, the system is triggered by executing service A and service B at step 1a and 1b. Service A completes its process successfully and sends successful message to the queue at step 2a. Service B completes its process unsuccessfully and sends unsuccessful message to the queue at step 2b. Response control service gets those messages from queue with same unique identifier at step 3. After that, response control service executes rollback steps for service A and sends a response '*completed unsuccessfully'* message to the client or the queue.

IV. Conclusion and Future Works

Understanding the final response of a system which is built by many microservices may be hard especially using asynchronous event driven architecture. Because microservices start their individual processes and complete independently, we must check all microservices to see if it is completed successfully or not in the queue. However, using separated and independent response control service, the system automatically checks the queue for responses of microservices and decides if there must be a rollback or not. Also, microservices are isolated from rollback scenarios, so they only focus on completing their processes and consume less CPU and memory at the same time.

On the other hand, there are some important points of using response control service. Response control service returns the system's output instead of any microservice in the system to the client. So, if a client starts the system by triggering a microservice in choreography Saga pattern, the client will take response from response control service instead of the microservice that triggered. So, the client needs to know that responses will be checked from another queue or a service. Also, if parallel execution method is used, the microservices must be totally independent from each other. Because they run in parallel, they must not take an input from another process's output.

Finally, checking and controlling the system for any error is easier than using traditional Saga pattern rollback procedures. In Saga pattern, every microservice must handle its rollback steps. Even in the orchestration approach, the controller service only handles triggering unsuccessful events to start a rollback process for a service. Also, the systems with response control service will work more efficiently because microservices execute only their processes and not the rollbacks.

Developing the response control service using a programming language and implementing to an existing choreography saga architecture system with high throughput may be the future work. Also making load test may be the future work. To see the performance difference between classic choreography saga architecture and response control service architecture, a load test with rollback scenarios may be implemented. Making the load test containing lots of error states will show the performance of the response control service.

V. References

[1] B. Andrade, S. Santos, A. Silva, "From Monolith to Microservices: Static and Dynamic Analysis Comparison", 2022.

[2] S. Newman, "Building Microservices: Designing Fine-Grained Systems", O'ReillyMedia Inc, 2015.

[3] W. Hasselbring, "Microservices for Scalability: Keynote Talk Abstract," Mar. 2016, pp. 133–134. doi: 10.1145/2851553.2858659.

[4] F. Tapia, M. Mora, W. Fuertes, H. Aules, E. Flores and T. Toulkeridis, "From Monolithic Systems to Microservices: A Comparative Study of Performance", 2020, Applied Sciences. 10. 5797. 10.3390/app10175797.

[5] K. Dürr, R. Lichtenthaler and G. Wirtz, "An Evaluation of Saga Pattern Implementation Technologies", 2021

[6] Vossen, G. (2009). ACID Properties. In: LIU, L., ÖZSU, M.T. (eds) Encyclopedia of Database Systems. Springer, Boston, MA. https://doi.org/10.1007/978-0-387-39940-9_831

[7] E. Daraghmi, C. Zhang, S. Yuan, "Enhancing Saga Pattern for Distributed Transactions within a Microservices Architecture", 2022.

[8] Y. Al-Houmaily, "Atomic commit protocols, their integration, and their optimizations in distributed database systems", 2010.

[9] Medjahed, B., Ouzzani, M., Elmagarmid, A.K. (2009). Generalization of ACID Properties. In: LIU, L., ÖZSU, M.T. (eds) Encyclopedia of Database Systems. Springer, Boston, MA. https://doi.org/10.1007/978-0-387-39940-9_736

[10] B. Lampson and D. Lomet, "A New Presumed Commit Optimization for Two Phase Commit", 1993.

[11] G. Samaras, K. Britton, A. Citron and C. Mohan, "Two-Phase Commit Optimizations in a Commercial Distributed Environment", 1995, 3. 325-360. 10.1007/BF01299677

[12] H. Garcia-Molina and K. Salem, "Sagas," ACM SIGMOD Record, vol. 16, no. 3, pp. 249–259, 1987.

[13] C. K. Rudrabhatla, "Comparison of Event Choreography and Orchestration Techniques in Microservice Architecture", 2018

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on November 10,2026 at 08:52:45 UTC from IEEE Xplore. Restrictions apply.